안녕하세요.
Javascript 강좌중인 Falcon 입니다.

오늘은 여러분들에게 유효범위에 대해서 알려드리려고 합니다. 그 전에 거의 모든 프로그래밍 언어의 변수는 크게 2가지 형태로 나뉘어져 있습니다. 아래의 표에 그 2가지 형태를 나타내고 있습니다.

전역변수
지역변수

다든 언어를 배우신 분들은 전역변수와 지역변수에 대한 개념을 알고 계실것이고, 그렇지 못한 분들은 아마 차이점을 모르실거라고 생각합니다. 지금부터 이 차이점을 알려드리도록 하겠습니다.

```
1   <!DOCTYPE html>
2   <html lang="en" dir="ltr">
3     <head>
4       <title>유효범위</title>
5       <meta charset="utf-8">
6     </head>
7     <body>
8       <script type="text/javascript">
9         var vscope = 'global';
10        function fscope(st) {
11          alert(st);
12        }
13        fscope(vscope);
14      </script>
15    </body>
16  </html>
```

제가 여러분들에게 함수를 가르쳐 드린 바로 위의 방식은 거의 모든 프로그래밍 언어에서 사용되는 방식입니다. 그런데 오늘 유효범위를 가르쳐드리면서 이 방식의 틀을 깨버릴 겁니다. Javascript는 9행에 선언된 변수가 전역변수 입니다. 전역변수라는 한 번 선언을 해두면 언제 어디서든 사용할 수 있는 변수를 뜻 합니다.
즉, 위에서 13행에 선언된 인자값이나 10행에 선언된 매개변수는 필요가 없습니다.

```
1   <!DOCTYPE html>
2   <html lang="en" dir="ltr">
3     <head>
4       <title>유효범위</title>
5       <meta charset="utf-8">
6     </head>
7     <body>
8       <script type="text/javascript">
9         var vscope = 'global';
10        function fscope() {
11          alert(vscope);
12        }
13        fscope();
14      </script>
15    </body>
16  </html>
```

9행에 선언된 변수가 11행에 실행결과 값으로 바로 들어가고 있습니다. 이게 가능한 이유는 9행에 선언된 변수가 전역변수이기 때문입니다. 그렇다면 그와 반대대는 지역변수에 대해서 알아보도록 하겠습니다.

```
1   <!DOCTYPE html>
2   <html lang="en" dir="ltr">
3     <head>
4       <title>유효범위</title>
5       <meta charset="utf-8">
6     </head>
7     <body>
8       <script type="text/javascript">
9         function fscope() {
10          var vscope = 'local';
11        }
12        alert(vscope);
13        console.log(vscope);
14      </script>
15    </body>
16  </html>
```

지역변수는 함수 내에 선언된 변수를 지역변수라고하는데 이 지역변수는 함수내에서만 사용할 수 있습니다. 즉, 위에서 12행과 13행은 실행되지 않습니다. 그 이유는 vscope라는 변수를 찾을 수 없기 때문 입니다.

```
▶Uncaught ReferenceError: vscope is not defined    index.html:12
    at index.html:12
```

윗 부분을 실행을 해보면 vscope라는 변수를 찾을 수 없다고 합니다. 전역변수로 선언을 하면 어디에서도 사용이 가능하지만 지역변수는 함수에 선언되면 함수 내부에서만 선언이 가능하다는 것을 잊지 마시기 바랍니다. 물론 전역변수와 지역변수에 대한 개념과 알아야할 부분들이 좀 더 있지만 지금은 이정도만 알고 계셔도 충분합니다.

```
1   <!DOCTYPE html>
2   <html lang="en" dir="ltr">
3     <head>
4       <title>유효범위</title>
5       <meta charset="utf-8">
6     </head>
7     <body>
8       <script type="text/javascript">
9         var vscope = 'global';
10        function fscope() {
11          var vscope = 'local';
12        }
13        fscope();
14        console.log(vscope);
15      </script>
16    </body>
17  </html>
```

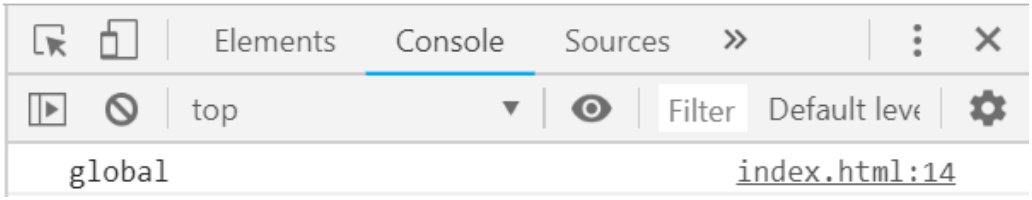
전역변수 vscope가 있다고해서 같은 이름으로 지역변수를 선언할 수 없는 것은 아닙니다. 위에서 14행 부분을 보면 vscope를 console로 출력하고 있습니다. 그런데 13행에서 함수를 호출하고 있습니다. 이 경우에는 결과값이 어떻게 될지 예상이 되시나요 ? 만약 'local' 이라고 생각을 한다면 틀렸습니다. 위에서 지역변수는 밖으로 나올 수 없다고 설명을 해드렸죠. 그렇기 때문에 함수를 호출했다고 해도 결과는 바뀌지 않습니다.

```
1   <!DOCTYPE html>
2   <html lang="en" dir="ltr">
3     <head>
4       <title>유효범위</title>
5       <meta charset="utf-8">
6     </head>
7     <body>
8       <script type="text/javascript">
9         var vscope = 'global';
10        function fscope() {
11          vscope = 'local';
12        }
13        fscope();
14        console.log(vscope);
15      </script>
16    </body>
17  </html>
```

전역변수를 선언할 때 9행을 보면 var라는 키워드를 적어주었습니다. 그런데 var라는 키워드를 쓰지 않아도 변수의 선언은 가능합니다. 이렇게 글로 설명하는 것 보다 한번 보여드리도록 하죠.

```
1   <!DOCTYPE html>
2   <html lang="en" dir="ltr">
3     <head>
4       <title>유효범위</title>
5       <meta charset="utf-8">
6     </head>
7     <body>
8       <script type="text/javascript">
9         vscope = 'global';
10        console.log(vscope);
11      </script>
12    </body>
13  </html>
```

위를 보시면 저는 분명 var라는 키워드를 선언하지 않고 변수의 선언을 해주었습니다. 그런데 이를 실행을 해보면



결과 값은 나오게 됩니다. 제가 이 부분을 설명드리는 이유는 var가 있고 없고의 차이점을 알아야 위의 코드를 분석할 수 있기 때문 입니다. 전역변수의 얘기라면 var가 있고 없고의 차이는 없습니다. 하지만 함수 내에 선언된 지역변수라면 얘기가 달라지게 됩니다. 다시 원래 소스코드로 한번 돌아와 봅시다.

```
1   <!DOCTYPE html>
2   <html lang="en" dir="ltr">
3     <head>
4       <title>유효범위</title>
5       <meta charset="utf-8">
6     </head>
7     <body>
8       <script type="text/javascript">
9         var vscope = 'global';
10        function fscope() {
11          vscope = 'local';
12        }
13        fscope();
14        console.log(vscope);
15      </script>
16    </body>
17  </html>
```

11행을 보면 var라는 키워드가 붙지 않았고, 함수 내부에 존재하기 때문에 지역변수라고 생각하면 큰일이 납니다. 11행에 var라는 키워드가 없이 선언된 vscope는 9행에 존재하는 전역변수를 의미하기 때문 입니다. 13행에서 함수를 호출하고 14행에서 vscope를 출력하면 'local'이라는 문자열을 출력해줍니다.

```
1   <!DOCTYPE html>
2   <html lang="en" dir="ltr">
3     <head>
4       <title>유효범위</title>
5       <meta charset="utf-8">
6     </head>
7     <body>
8       <script type="text/javascript">
9         vscope = 'global';
10        function fscope() {
11          var vscope;
12          vscope = 'local';
13        }
14        fscope();
15        console.log(vscope);
16      </script>
17    </body>
18  </html>
```

9행에도 vscope가 var라는 키워드를 통해서 선언되어 있고, 11행에도 vscope가 있을 경우에는 함수 내부에서 가까운 11행의 vscope를 따라가게 됩니다. 즉, 위의 경우라면 15행의 출력 결과는 'global'이 됩니다. var라는 키워드를 선언했을 경우와 선언하지 않았을 경우의 차이점을 확실히 알고 넘어가시기 바랍니다.

```
1   <!DOCTYPE html>
2   <html lang="en" dir="ltr">
3     <head>
4       <title>유효범위</title>
5       <meta charset="utf-8">
6     </head>
7     <body>
8       <script type="text/javascript">
9         function func() {
10          var i = 0;
11        }
12        for(var i = 0; i < 5; i++)
13        {
14          func();
15          document.write(i + " ");
16        }
17      </script>
18    </body>
19  </html>
```

위의 코드는 정상적으로 실행이 가능한 코드 입니다. 10행에서 var라는 키워드가 붙어 있기 때문 입니다.

0 1 2 3 4

실행을 해보시면 이렇게 0부터 4까지 출력되는 모습을 볼 수 있지만 만약 10행의 부분에 var라는 키워드가 붙지 않았을 경우에는 얘기가 달라지게 됩니다.

```
1   <!DOCTYPE html>
2   <html lang="en" dir="ltr">
3     <head>
4       <title>유효범위</title>
5       <meta charset="utf-8">
6     </head>
7     <body>
8       <script type="text/javascript">
9         function func() {
10          i = 0;
11        }
12        for(var i = 0; i < 5; i++)
13        {
14          func();
15          document.write(i + " ");
16        }
17      </script>
18    </body>
19  </html>
```

위의 코드는 10행에 var라는 키워드가 붙지 않았습니다. 이 경우에는 무한 반복이 일어나게 됩니다. 왜? 14행에서 함수를 호출하게 되면 12행에 선언한 var i라는 전역변수가 0으로 초기화 되어 버리기 때문 입니다. 위의 경우 아주 간단한 소스코드라서 쉽게 오듀를 발견할 수 있지만 이미 수천 수만 코드가 진행되어 버린 코드에서 이런 오듀를 찾기는 쉽지 않습니다. 다든 언어를 배운 분들이라면 여기서 혼란스러울 수 있을텐데 Javascirpt는 다든 언어와 달리 매우 자유로운 면을 가지고 있는 언어 입니다. 보통 for문 안에 선언된 변수들은 for문이 종료됨과 동시에 사라져버립니다. 그런데 Javascript는 for문이 종료되어도 변수가 사라지지 않습니다.

```
1 public class Test1 {
2     public static void main(String[] args)
3     {
4         for(int i = 0; i < 5; i++)
5         {
6             int sum = i++;
7         }
8         System.out.println(i);
9     }
10 }
```

JAVA

이 코드 언젠가 여러분들도 배우게 될 코드이지만 여기서 중요한 것은 4행 부분의 int i의 선언이 8행으로 나오지

못합니다. JAVA에서는 for문 안에 선언된 변수는 for문이 종료됨과 동시에 사라져버리기 때문 입니다.

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main(void) {
6       for (int i = 0; i < 5; i++) {
7           int sum = i++;
8       }
9
10      cout << i << endl;
11
12      return 0;
13  }
```

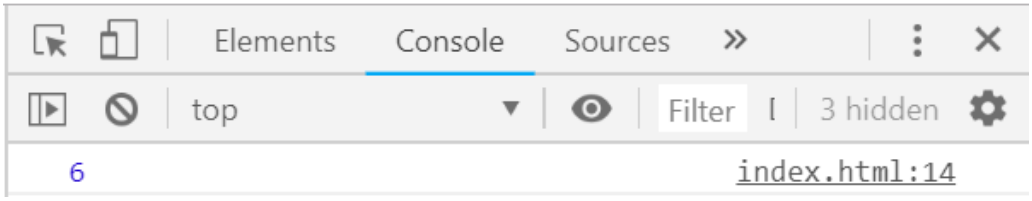
C++

이건 C++언어 역시 마찬가지 입니다. for문안에 선언된 int i를 10행 부분에서 출력을 하지 못합니다. 그런데 Javascript에서는 이것이 가능 합니다. 그 이유를 설명해드리자면 Javascript는 밖에 선언된 모든 변수를 전역변수로 인식을 해버리는 것 같습니다.. 그럼 한번 확인해보도록하죠.
(이 부분은 정확하지 않습니다. 다만 여러 실험을 통해서 실행을 해본 결과가 전역변수로 인식했습니다.)

```html
<!DOCTYPE html>
<html lang="en" dir="ltr">
  <head>
    <title>유효범위</title>
    <meta charset="utf-8">
  </head>
  <body>
    <script type="text/javascript">
      for(var i = 0; i < 5; i++)
      {
        var sum = i++;
      }
      console.log(i);
    </script>
  </body>
</html>
```

Javascript

언어별로 사용방법을 보면 크게 다르지 않다는 것을 알 수 있습니다. 한 언어를 마스터를 하면 다든 언어를 배우는 것은 어렵지 않습니다. 다만 언어마다 조금씩 사용방법이 다를 뿐 입니다. 여기서 중요한 것은 Javascript에서는 이 소스코드가 실행이 됩니다. 한번 확인해보도록 하죠.



위의 코드가 정상적으로 실행이되어 6이 뜨는 것을 확인할 수 있습니다. Javascript는 for문이 종료된 후에도 변수가 전역변수로 그대로 남아 있습니다. 그렇기 때문에 함수에 변수를 선언할 때에는 신중해야합니다.

```
1   <!DOCTYPE html>
2   <html lang="en" dir="ltr">
3     <head>
4       <title>유효범위</title>
5       <meta charset="utf-8">
6     </head>
7     <body>
8       <script type="text/javascript">
9         var i = 5;
10
11        function a() {
12          var i = 10;
13          b();
14        }
15
16        function b() {
17          console.log(i);
18        }
19      </script>
20    </body>
21  </html>
```

글을 잘 읽어주시고 이해를 하셨으면 좋겠습니다. 일단 9행에 존재하는 var i는 전역변수 입니다. 그런데 12행에 존재하는 var i는 지역변수 입니다. 그리고 13행을 보면 b라는 함수를 호출해서 i의 값을 출력하고 있습니다. 이 경우에 과연 17행에 출력되는 i의 값은 전역변수 일까요? 아니면 지역변수 일까요 ? 정답은 전역변수 입니다.

이는 잘 생각을 해보면 쉽게 알 수 있습니다. 9행에 존재하는 var i는 전역변수이기 때문에 b라는 함수에 갈 수 있지만 a라는 함수 내에 선언된 12행 부분의 var i는 지역변수이기 때문에 b라는 함수에 갈 수 없습니다.

```
function a() {
  var i = 10;
  b(i);
}

function b(i) {
  console.log(i);
}
```

만약 a라는 함수의 var i의 값을 출력을 하고 싶다면 인자값과 매개변수를 통해서 전달을 해주면 되겠습니다.
변수에 대한 부분들은 이정도로 정리를 해두도록하고 다음 시간에는 객체에대한 애기를 해보겠습니다.

이것으로 [Javascript 강좌] 제 15강 변수의 유효범위 편을
마치도록 하겠습니다.